Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 77–86

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

An Effective Deployment of Controllers in Software-Defined Networks

Yasmine.Sk[a], Prabu.U[a]*, Sri Deepak Phaneendra.Y[a], Geetha.V[b]

[a]Department of CSE, Velagapudi Ramakrishna Siddhartha Engineering College, Vijayawada-520007, India
[b]Department of IT, Puducherry Technological University, Puducherry-605014, India
[a]*uprabu28@gmail.com

Abstract

Network performance has become a crucial consideration in ensuring a flawless user experience in the era of data-intensive applications and cloud computing. Traditional Networks are not suitable for solving such issues, so Software-defined networking (SDN) has evolved as a game-changing technology, promising network flexibility, scalability, and centralized control. However, some challenges remain open to the research, and one of the most popularly known challenges is controller problem placement (CPP). Latency is also a major consideration in the network. However, implementing low-latency communication in SDN systems remains a difficult issue, especially as networks become more complicated. In the realm of SDN, where centralized control meets distributed data planes, minimizing latency between switches and controllers emerges as a pivotal challenge. This article delves into the critical concept of optimizing controller placement within SDN networks to minimize latency, with a primary focus on considering the switch-controller distance. The proposed work uses a Linear programming-based approach to identify the optimal controller location in the network by considering the switch-controller distance and the latency is calculated. Further, a comparison of the proposed work with a heuristic-based approach is performed which shows that the proposed work performs well in terms of latency.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords— SDN; Controller placement problem (CPP); Latency; Controller; Linear programming.

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.197

1. Introduction

Network administration has become a significant issue in the modern world as a result of the rise in communication devices, as traditional networks are inadequate to handle such complex networks. SDN was developed as a solution to overcome this problem. In traditional networking, the data layer and the control layer are integrated which contains the hardware in the network such as switches, routers, etc. In SDN the data layer and control layer are separated where the data layer contains all the switches and the control layer contains the controllers as a result increases the network efficiency and performance [1].

1.1. SDN Architecture

The three layers [2] that make SDN architecture (Fig 1) are

1. Data Layer: The foundational layer in the architecture of SDN includes forwarding devices like routers and switches.
2. Control Layer: The middle layer in the architecture of SDN which consists of the software units called controllers which manage and control the SDN network.
3. Application Layer: The topmost layer in the architecture of SDN which consists of SDN applications deployed on the network.

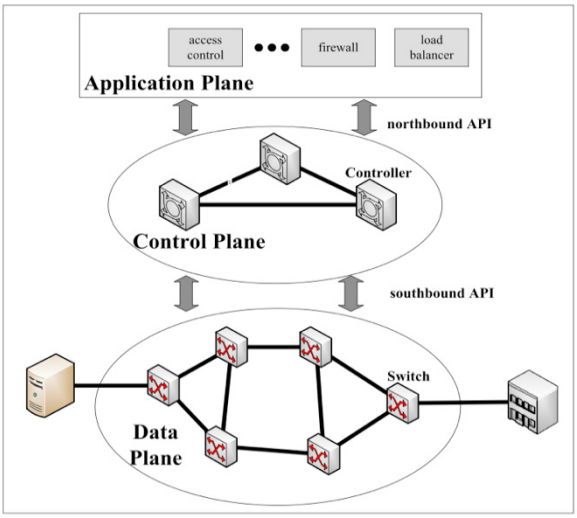


Fig. 1. SDN Architecture [2]

SDN consists of software units called controllers which serve as the core of an SDN. The software component acts as the brain of the centralized. Using protocols like OpenFlow or NETCONF, it communicates with network switches and routers. Initially, only a single controller is considered to simplify the network, but it becomes difficult to handle the network with a single controller if the network expands [3].

Since controllers are the core of the network choosing the optimal position and right count of the controllers plays a major role and this problem is popularly known as the CPP. In CPP, the user should provide some objectives to be considered while optimizing the placement, such as lowering the latency, boosting fault tolerance, and optimizing energy utilization [4]. For a given SDN network, there will be three major concerns concerning CPP [5],

1. count of the controllers
2. location of the controllers
3. assigning switches to controllers

As the control logic is divided from forwarding devices and all the functionalities in the network are done by the communication between the switches and controllers, therefore latency plays an important role in the SDN network [6]. Network latency is of three types, transmission, propagation, and processing latency. The transmission latency is generally a constant value with the networking devices belonging to the same network, propagation latency mainly depends on the node-node distance and the processing latency affects the load and processing capabilities of the network [7],[8]. Therefore, in CPP only propagation latency and processing latency are taken into consideration. Again, the latency can be divided into various levels by considering the framework of SDN Intra-domain and Inter-domain latency. Intra-domain latency defines the latency between the components in the same domain of layer and Inter-domain latency defines the latency between the components of different domains [3].

Since latency plays an important role a Linear programming (LP) model to reduce the latency between the switches and controllers by placing the controllers in optimal locations is proposed and calculating the latency of the network before and after optimization.

The contributions of the article revolve around addressing key aspects of the Controller Placement Problem (CPP) in Software-Defined Networks (SDNs). First, the aim is to determine the most optimal locations for controllers, strategically positioning them within the network to maximize their utility and efficiency. Moreover, the primary focus lies in calculating and minimizing latency in data propagation between controllers and network nodes.

The article's structure is as follows: Section 2 covers the related research, Section 3 presents the proposed work, Section 4 outlines the experimental evaluation, and Section 5 concludes the article.

2. Background

The issue of traffic matrix estimate has long been researched. This section deals with the survey of various approaches so far done to estimate an accurate traffic matrix. Mohanty, Sagarika, et al [9] proposed an efficient genetic algorithm to find the optimal controllers positions which reduce cost. The proposed project aims to optimize controller placement in a network by employing random placement initially, followed by switch allocation based on shortest distances. It seeks to enhance this process using a genetic algorithm and compare its performance with simulated annealing and random placement methods. The project's goal is to achieve minimal network latency while reducing overall costs, and it will be validated using real datasets from the topology zoo.

Liao, Chenxi, et al. [10], proposed an innovative approach to address the Distributed CPP in SDN. The proposed approach, called MODECP (Multi-Objective Based Approach for Distributed CPP), leverages multi-objective optimization techniques to find a set of Pareto-optimal solutions that offer trade-offs between conflicting objectives. The evaluation of the MODECP approach is conducted through simulations using real-world network topologies and traffic patterns. By optimizing the placement of controllers, MODECP aims to minimize network latency, which can result in improved network performance. The limitation of this approach is it is computationally intensive, especially for large-scale networks.

To enhance network connectivity and flexibility between controllers and switches, [11] introduced a novel graph theory. Subsequently, an optimal controller is selected from a pool of controllers through the utilization of a firefly optimization algorithm (FA), leading to enhanced network performance. To reduce communication latency, the placement of multiple controllers is strategically determined by considering both the positions and distances. This multi-controller placement strategy is executed using a combination of HAS and PSO algorithms. The limitation of this approach is the article may make certain assumptions in terms of network topologies, traffic patterns, or other parameters. These assumptions may not hold in all real-world scenarios, which could limit the generalizability of the proposed algorithm.

In the research on controller placement within a network, the innovative Naked Mole Rat (NMR) algorithm was introduced by A. Sapkota, R. Dawadi B. Babu, and R. Joshi C. Shashidhar [12]. The primary objective of this algorithm is to optimize controller positions by considering latency for both the controllers and controller-to-switch, with a specific emphasis on load balancing between the controllers. The NMR approach is employed to achieve optimal controller placement while ensuring load balance, resulting in a reduction of latency for both the controllers and controller-to-switch. The limitation of this approach is the Naked Mole-Rat Algorithm (NMRA) scales as the network size or complexity increases. Some algorithms that work well for small networks may struggle to handle large-scale SDN deployments.

In the effort to address challenges related to task completion delays and inefficient utilization of computational resources, a technique aimed at resource allocation was introduced by Chunlin Li, Kun Jiang, and Youlong Luo [13], primarily stemming from irregular edge server allocation. This method relies on considerations related to constraints, task delay, and reliability. To achieve efficient resource allocation, a heuristic ant colony (HAC) algorithm is employed, providing a viable solution to optimize the distribution of computing resources. As it's a heuristic algorithm it is not sure that it can be adaptable to real-world scenarios.

Seyedkolaei, Ali Abdi, et al. [14], proposed an approach for deploying SDN controllers in a cost-effective and survivable manner. The approach considers the possibility of controller failures and strives to maintain network operability, even if failures occur. The authors define a set of decision variables that finds the controllers placement and the capacity of the links connecting the controllers to the switches. They construct an optimization challenge designed to minimize the overall deployment cost of controllers while also guaranteeing the network's continued operation in the event of multiple failures. The limitation of this proposed work is it is not suitable for networks where traffic patterns are often dynamic.

Moradi, A. et al., [15], proposed an Iterated Local Search (ILS) algorithm to address the CPP in SDN. The objective is to reduce the required number of controllers while upholding specific performance criteria for the network, including factors such as latency, throughput, and fault tolerance. The ILS algorithm iteratively searches for better solutions by modifying the placement of controllers and evaluating the resulting network performance. During each iteration, the algorithm chooses a specific set of switches for allocation to a controller and subsequently assesses the network's performance based on this updated allocation. If the performance is improved, the new assignment is accepted, and the search continues with the new assignment. Otherwise, the algorithm backtracks and explores other possibilities. The limitation of this approach is computational complexity. The limitation of this proposed work is it is very much suitable for large-scale networks but complex for small-scale networks.

Kanodia, Khushboo, et al. [16] proposed a methodology, CCPGWO to find the ideal position of controllers in a network to reduce the worst-case latency in the event of a link loss. Within this approach, the assumption is made that every node is interconnected by multiple links, thus ensuring alternative pathways in the event of a failure. Additionally, it is presumed that all nodes are equipped with OpenFlow capabilities, providing an advantage for the deployment of controllers at any node. The limitation of this proposed work is its convergence rate is poor and its local search ability is not too good.

Fan, Yuqi, Lunfei Wang, and Xiaohui Yuan. [17] proposed a novel multi-objective technique, the RALO algorithm to minimize the switch-controller latency in both cases with and without link failure. In this context, where each switch is allocated to a sole controller, the likelihood of having numerous network connections is quite low. Under these conditions, an assumption is made that, at most, a single link failure might occur. This stems from the fact that, with each switch exclusively assigned to one controller, it is not possible to divide switch requests for concurrent processing by multiple controllers; maintaining a single-controller approach ensures data accuracy and prevents conflicts.

An algorithm that blends the strengths of nature-inspired optimization methods, specifically Manta-Ray Foraging Optimization (MRFO) and Salp Swarm Algorithm (SSA), with network partitioning was introduced in [18] by Nasrin, Firouz et al. This unique meta-heuristic-based approach was developed to address the challenges of dynamic load distribution and latency in the context of CPP within SDN. This CPP method accounts for key factors, including propagation delay from controllers to switches, dynamic traffic loads, and the optimal distribution of switches to its most suitable controllers. Notably, in large-scale SDN networks, propagation delay plays a pivotal role in network performance, making its optimization a key objective for the Hybrid Controller Placement Algorithm (HCPA), which employs network segmentation techniques to strategically reduce latency by selecting an optimal group of switches to serve as controllers. The limitation of this work is it is complex to apply the work for small networks.

An innovative resource allocation approach designed to tackle two pressing issues, namely task completion delays and the inefficient utilization of computational resources due to irregular edge server allocation, was introduced in [19] by Chunlin Li, Kun Jiang, and Youlong Luo. This method is rooted in considerations involving constraints, task delay, and reliability. To enhance the allocation of computing resources, the authors employed a heuristic ant colony (HAC) algorithm, which provides an efficient solution. Additionally, the proposed method aims to rectify communication latency issues related to network link failures between controllers and switches, along with addressing communication challenges among the controllers themselves. The major limitation of this work it's inadaptability to real-time networks.

Schutz, Gabriela, and Jaime A. [20], proposed a three-step approach that takes into account various factors such as topology of the network, traffic patterns, and controller capabilities. The first step involves modeling the topology of the network and traffic patterns, while the second step involves evaluating potential controller placement configurations using a set of metrics such as latency and controller load. In the third step, the authors use a genetic algorithm to search for the optimal placement configuration based on the results of the previous steps. The limitation of this work is it is not sure that it will adapt to real-time networks.

3. Proposed Work

The proposed methodology involves utilizing the topology of the network as an input to address the CPP. The main aim of this approach is to reduce latency between switches-controllers. The proposed work uses a linear programming-based method to identify the optimal locations for controllers within the network. Subsequently, the controller-switches latency is calculated as part of the final outcome.

3.1 Workflow Diagram

Fig 2 shows a workflow diagram showing how the project flows from start to end. The Network agent takes a CSV file as a data set. The controller number and the maximum number of switches a controller can handle will be defined by the Network agent. The controller placement problem is formulated as a minimization problem. Define the decision variable and Objective function. Define the constraints. The optimal locations of controllers will be calculated.

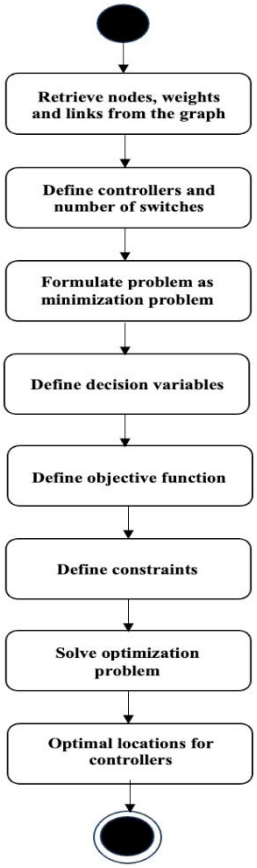


Fig. 2. Workflow diagram

3.2 Problem Formulation

The network's structure is described using a graph, denoted as $G(N, C)$, where N represents the network nodes, and C represents the connections linking these nodes. In this context, $N = |N|$ signifies the total count of nodes, and $L = |L|$ signifies the total number of connections. The variable G_{ij} represents the latency between the nodes in the network, while s denotes the maximum capacity of switches that a controller can manage. The variable x_{ij} is set to 1 when a switch i is assigned to a controller j or 0 otherwise. The primary objective of the problem is to minimize the latency between switches and controllers, and the corresponding objective function is expressed as follows.:

$$\sum_{i=1}^{n}\sum_{j=1}^{m} G_{i,j}\, x_{i,j} \tag{3.1}$$

Table 1 outlines the notations used in the proposed work.

Table 1. Notation Table

Notation	**Description**
n	Count of Nodes
m	Count of controllers
c	Maximum count of switches a controller can hold
$L[]$	List to hold the adjacency matrix
$X[i,j]$	1 denotes that switch i is allocated to controller j, while 0 indicates otherwise.
$Y[j]$	1 if controller j is used, and 0 otherwise
Z	Set of Controllers
el	Latency Before Optimization
al	Latency after Optimization
L	Reduction in Latency

3.2 Methodology

3.2.1 Solving Objective Function

The Solving Objective Function procedure aims at solving the CPP: to find the optimal controller location in the network, to reduce the overall latency in the network. The solution to the CPP is depicted by using linear programming.

Algorithm 1 represents the procedure flow in a step-by-step manner for finding the optimal placements of the controllers. Initially, constraints are established to ensure the exclusive allocation of each switch to a single controller while also ensuring that the count of switches allocated to any given controller remains within the specified maximum capacity. Following the definition of these constraints, the objective function for the CPP is formulated. This objective function aims to minimize the overall network latency, computed by multiplying the weight associated with each network edge by the respective controller placement variable. Finally, the linear programming problem is formulated using the LP Problem class from the pulp library. Decision variables are defined, including x variables representing

the placement of controllers and y variables representing whether a controller is active which are used to determine the optimal controller placement.

Algorithm 1: Solving optimization problem using Linear Programming

Input: $\boldsymbol{n, m, c}$

Define Empty List G[] to hold the adjacency matrix

Define Decision variables

$x[i,j]$

$y[j]$

Define constraints

$$\sum_{j=1}^{m} x_{i,j} = 1 \; \forall \in S$$

$$\sum_{i=1}^{n} x_{i,j} \leq cy_i$$

Define optimization problems

Define objective function

$$minimize \sum_{i=1}^{n}\sum_{j=1}^{m} G_{i,j}\, x_{i,j}$$

```
Create a Set Z
for all j ∈ m do
    for all i ∈ n do
        if value of x[i,j] == 1
            Add i to Z
        end if
    end for
end for
```

Output: Z

3.1.2 Calculating Latency

Calculating Latency which is the final procedure involves calculating the latency in the network. Here calculation of latency is mainly dependent on finding the shortest path length. Algorithm 2 iterates over each switch node in the network graph. For each switch node, it determines the closest controller node based on the shortest path length. By finding the controller node that minimizes the shortest path length from each switch node, the function determines the controller that offers the lowest latency for communication with that switch. It accumulates the latencies between switches and its corresponding controllers. Finally, total network latency is the sum of all individual latencies between switches and its corresponding controllers. This value indicates the overall network performance in terms of latency.

Algorithm 2- Latency Calculation

Input: $\boldsymbol{G}[]$

1: Calculate Latency before Optimization
2: for all i ∈ n do
3: for all $j \in m$ do
4: $el += sum(G[i][j])$
5: end for
6: end for
7: Latency after Optimization
8: $al = value\ of\ the\ objective\ function$
9: Reduction in latency
10: $L = el - al$
Output: *L*

4. Experimental Evaluation

In this section, the potential improvements of the proposed method on the CPP are evaluated. This section is structured as follows:

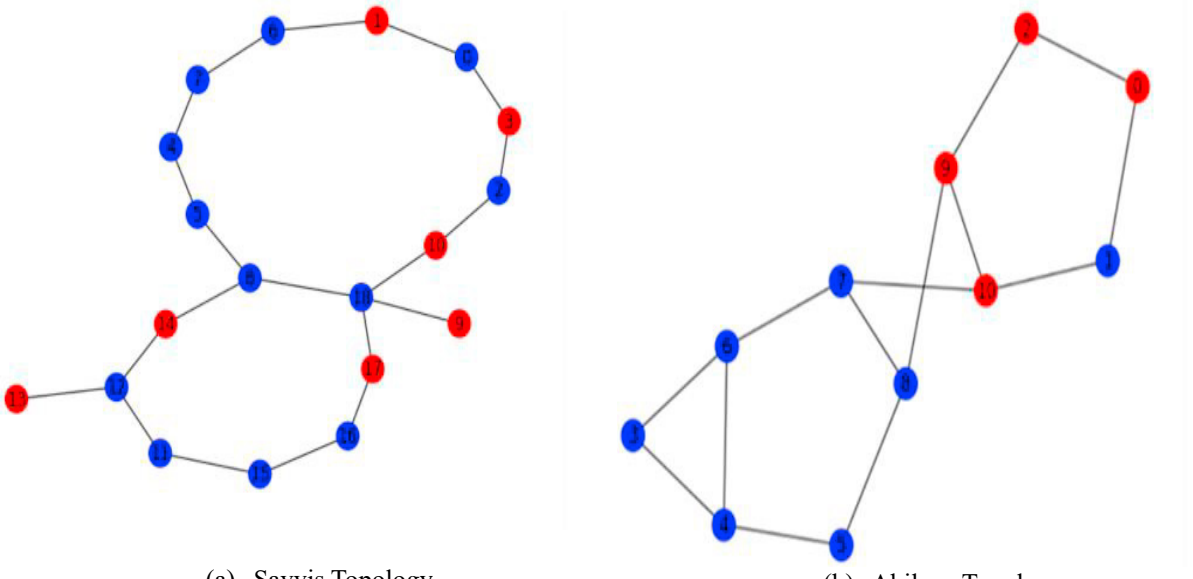
i. First, describe the experimental setup data used in the evaluation process, and then.
ii. Perform a comparison by evaluating the total network latency and the latency achieved through this method.

4.1 Experimental Setup

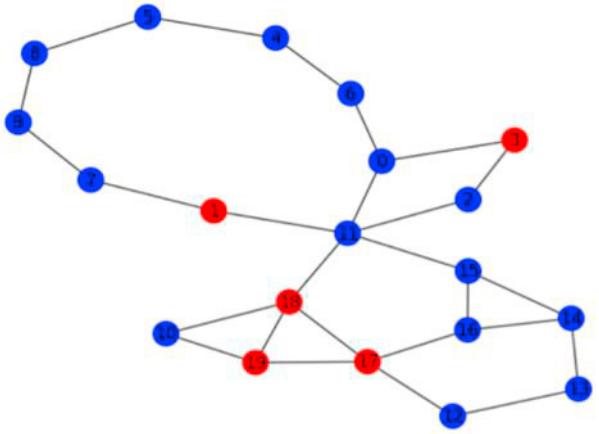
The experiment is carried out with GPU T4 with 8GB of RAM. Python 3.10 is used as a programming language. Savvis, Abilene, and Oxford are used for experimental evaluation.

4.2 Performance Evaluation

In this section, the performance of the proposed CPP optimization method is evaluated across three different network topologies: Savvis, Oxford, and Abilene. The effectiveness of the approach is analyzed by comparing the network latency before and after optimization and visually representing the results. The optimal placement of controllers for the different network topologies is given in Figure 3.



(a) Savvis Topology (b) Abilene Topology



(c) Oxford Topology

Fig. 3. Optimal Placement of Controllers

The obtained results demonstrate a significant reduction in network latency after applying the controller placement optimization method to all three network topologies. In the Abilene, Oxford, and Savvis. And achieved a reduction in network latency. This reduction is a clear indication of the effectiveness of the approach in optimizing controller placement. The results are with the baseline network latency before optimization. In all cases, the proposed method outperformed the baseline, showcasing its superior ability to reduce network latency.

The Heuristic approach calculates network latency by directly measuring the physical distances between switches and controllers in a 2D space. This approach considers only geometric factors. The Linear programming approach formulates network latency as edge weights in a graph and uses Linear Programming to optimize controller placement. It offers a more versatile method, allowing network latency to represent various factors beyond simple physical distance, making it adaptable for a broader range of network optimization challenges. The outcomes of these approaches are summarized in Table 2 for comparison.

Table 3 provides the Optimal position of Controllers and the value of latency for three different network topologies: Savvis, Abilene, and Oxford. These latencies were assessed both before and after optimization.

Table 2. Comparison of Latency for Heuristic Approach and Linear Programming Approach

Topology	**Savvis**	**Abilene**	**Oxford**
Heuristic	69	81.95	18
LP	52	64	36

Table 3. Experimental Analysis of Proposed work for three different topologies

Topology	**Savvis**	**Abilene**	**Oxford**
Optimal Position of Controllers	{1,3,9,10,13,14,17}	{0,2,9,10}	{1,3,7,18,19}
Latency before optimization	396	573	168
Latency after optimization	52	64	36
Reduction in latency	344	509	132

5. Conclusion and Future work

This article discussed an effective deployment of controllers in SDN which can enhance the network performance as CPP is a major concern in SDN. In the article, a Linear programming-based approach that minimizes the latency between switches-controllers by finding the distance between the switch-controller is proposed and finally, graphs are produced for every topology taken by marking the positions of the controllers in the network. And finally, the proposed work is compared with the heuristic-based approach which shows that the proposed work performs well in terms of latency. In the proposed work, switch-controller latency has been calculated, and as part of future work, controller-controller latency will be determined.

References

[1] Seyedkolaei, Ali Abdi, et al. "Cost-effective survivable controller placement in software-defined networks." IEEE Access 9 (2021): 129130-129140.

[2] Ahmad, Suhail, and Ajaz Hussain Mir. "Scalability, consistency, reliability, and security in SDN controllers: a survey of diverse SDN controllers." Journal of Network and Systems Management 29 (2021): 1-59.

[3] Lu, Jie, et al. "A survey of controller placement problem in software-defined networking." IEEE Access 7 (2019): 24290-24307.

[4] Mamushiane, Lusani, Joyce Mwangama, and Albert A. Lysko. "Given a SDN topology, how many controllers are needed and where should they go?." 2018 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). IEEE, 2018.

[5] Heller, Brandon, Rob Sherwood, and Nick McKeown. "The controller placement problem." ACM SIGCOMM Computer Communication Review 42.4 (2012): 473-478.

[6] J. F. Kurose, Computer Networking: A Top-Down Approach Featuring the Internet. London, U.K.: Pearson, 2005.

[7] G. Wang, Y. Zhao, J. Huang, and Y. Wu, ‘‘An effective approach to controller placement in software defined wide area networks,’’ IEEE Trans. Netw. Service Manage., vol. 15, no. 1, pp. 344–355, Mar. 2018.

[8] T. Zhang, P. Giaccone, A. Bianco, and S. De Domenico, ‘‘The role of the inter-controller consensus in the placement of distributed SDN con- trollers,’’ Comput. Commun., vol. 113, pp. 1–13, Nov. 2017.

[9] Mohanty, Sagarika, et al. "Metaheuristic techniques for controller placement in Software-Defined networks." TENCON 2019-2019 IEEE Region 10 Conference (TENCON). IEEE, 2019.

[10] Liao, Chenxi, et al. ”MODECP: A Multi-Objective Based Approach for Solving Distributed Controller Placement Problem in Software Defined Network.” Sensors 22.15 (2022): 5475.

[11] Radam, Neamah S., Sufyan T. Faraj Al-Janabi, and Khalid Sh Jasim. "Multi-controllers placement optimization in sdn by the hybrid hsa-pso algorithm." Computers 11.7 (2022): 111.

[12] Sapkota, A., B. Babu R. Dawadi, and C. Shashidhar R. Joshi. "Multi-Controller Placement Optimization Using Naked Mole-Rat Algorithm over Software-Defined Networking Environment." Journal of Computer Networks and Communications 2022 (2022).

[13] Li, Chunlin, Kun Jiang, and Youlong Luo. "Dynamic placement of multiple controllers based on SDN and allocation of computational resources based on heuristic ant colony algorithm." Knowledge-Based Systems 241 (2022): 108330.

[14] Seyedkolaei, Ali Abdi, et al. ”Cost-effective survivable controller placement in software-defined networks.” IEEE Access 9 (2021): 129130-129140.

[15] Moradi, A., A. Abdi Seyedkolaei, and Seyed A. Hosseini. ”Controller placement in software defined network using iterated local search.” Journal of AI and Data Mining 8.1 (2020): 55-65.

[16] Kanodia, Khushboo, et al. "CCPGWO: A meta-heuristic strategy for link failure aware placement of controller in SDN." 2020 International Conference on Inventive Computation Technologies (ICICT). IEEE, 2020.

[17] Fan, Yuqi, Lunfei Wang, and Xiaohui Yuan. "Controller placements for latency minimization of both primary and backup paths in SDNs." Computer Communications 163 (2020): 35-50.

[18] Firouz, Nasrin, et al. "A novel controller placement algorithm based on network portioning concept and a hybrid discrete optimization algorithm for multi-controller software-defined networks." Cluster Computing 24 (2021): 2511-2544.

[19] Li, Chunlin, Kun Jiang, and Youlong Luo. "Dynamic placement of multiple controllers based on SDN and allocation of computational resources based on heuristic ant colony algorithm." Knowledge-Based Systems 241 (2022): 108330.

[20] Schutz, Gabriela, and Jaime A. Martins. ”A comprehensive approach for optimizing controller placement in Software-Defined Networks.” Computer Communications 159 (2020): 198-205.